PRESENTATION COMPANION · IMPLEMENTATION STATUS

AIRLOCK

An honest account of the current prototype: what was designed, what is implemented, what was deliberately compromised, and what still needs proof.

Prepared for presentation use
Repository reviewed locally on 12 October 2026
Scope: Dev Server Guard · MCP Watch · Secret Gate

Read this alongside *Airlock — Master Project Document*. The master document explains the intended architecture and rationale; this report records the evidence available in the current codebase.

1. Executive statement

Airlock is a local-first security prototype for three developer workflows: exposing a local development server, using MCP-connected AI tools, and handling plaintext environment files. The project is credible as a prototype because it contains executable TypeScript components, a CLI, focused regression tests, packaging controls, and an evidence-based demonstration flow.

> **The accurate one-sentence claim:** Airlock provides a loopback-only request filter for a wrapped development server, a passive MCP stdio monitor, and a local in-memory secret broker with explicit source-file deletion.

It is **not** a production-certified security product, a complete host security boundary, an MCP enforcement engine, or a hardware-backed secret vault. The distinction is important: the master document describes a desired direction; the repository proves only the behaviors listed in this report.

Why the three parts belong together

- **Dev Server Guard:** reduces browser-to-loopback DNS-rebinding exposure by accepting only local Host/Origin/Referer authorities.
- **MCP Watch:** makes suspicious tool-path requests visible without changing MCP message content.
- **Secret Gate:** removes routine plaintext-file access after a deliberate transfer into process memory.

2. Architecture as implemented

Component	Actual implementation	Security posture
Dev Server Guard	Node HTTP reverse proxy bound to `127.0.0.1`. It starts a child dev-server command with localhost-oriented environment variables and forwards HTTP/WebSocket traffic to an internal target port.	**Rejecting.** Requests with invalid Host, Origin, Referer, or duplicate sensitive headers are dropped before forwarding.
MCP Watch	Node child-process wrapper. It copies stdin to the wrapped server and wrapped stdout to stdout. It parses newline-delimited JSON-RPC tool calls only for observation.	**Passive/fail-open.** It warns for traversal/sensitive paths but intentionally does not block them.
Secret Gate	In-memory map loaded from a dotenv file and served through a local Unix-domain socket (named pipe path on Windows).	**Local convenience boundary.** Request sizes, key format, timeouts, socket permissions, and explicit deletion are enforced.

Trust boundaries

- Guard trusts only loopback authorities; it does not authenticate an application user.
- MCP Watch trusts the wrapped MCP server and observes only the JSON-RPC framing it understands.
- Secret Gate relies on OS user/process isolation and socket filesystem permissions; it does not authenticate a hostile same-user process.

3. Delivered capabilities and evidence

Dev Server Guard

- Strict host parser accepts only `localhost` or `127.0.0.1`, with an optional numeric port from 1–65535.
- Regression coverage rejects `localhost:3000@evil.com`, suffix tricks, duplicated ports, invalid ports, missing Host, and hostile Origin/Referer values.
- Origin and Referer are parsed as URLs, require HTTP/HTTPS local hosts, and reject credentials.
- HTTP and WebSocket upgrade requests are checked before proxy forwarding.

Secret Gate

- Uses `dotenv` parsing for normal quotes, CRLF, comments, whitespace, and values containing equals signs.
- Validates environment-variable names and bounds requests to 128 names / 16 KiB.
- Uses a per-user runtime directory with Unix directory mode 0700 and socket mode 0600.
- Probes a pre-existing socket before removal and refuses to replace a non-socket object.
- Loads and binds before deletion. Deletion requires `--delete-source`.

MCP Watch

- All visual output and logs use stderr; stdout is reserved for protocol bytes.
- Monitors common path argument names and logs traversal or sensitive-path alerts.
- Buffers incomplete newline-delimited JSON messages up to 1 MiB to reduce memory-abuse risk.
- Explicitly labels itself *MONITORING ONLY*; it does not claim to block an operation.

4. Compromises and improvised decisions

These are deliberate prototype trade-offs, not hidden defects. Present them as engineering judgment: each buys simplicity or compatibility at the cost of coverage.

Decision / compromise	Reason	Consequence
MCP Watch is passive, not enforcing.	Blocking malformed or uncertain tool traffic can break an IDE workflow and requires protocol-specific behavior.	A warning is not protection. A malicious operation may still occur.
Newline-delimited JSON-RPC inspection.	Simple, transparent handling for common stdio MCP servers.	Other framing/transports are not proven. Do not market universal MCP coverage.
Hostname allowlist limited to localhost/IPv4 loopback.	Eliminates ambiguous authority parsing and keeps the security boundary comprehensible.	IPv6 loopback and custom local domains are unsupported unless intentionally added and tested.
Child server receives common HOST/PORT variables.	Works with many development servers without shell command construction.	Not universal: a server that ignores those variables can bind differently or fail to start.
Local Unix socket for secrets.	Simple no-network IPC with restrictive permissions.	It cannot defend against same-user malware, memory inspection, swap, backups, snapshots, or debugging.
Ordinary file unlink for optional deletion.	Portable, explicit behavior.	It is not secure erasure; data can remain recoverable on modern filesystems.

5. Validation and test truth

> **Commands successfully run on the reviewed workspace:**
> `npm run build` · `npm test` · `npm run lint` · `npm run test:coverage` · `npm pack --dry-run`

Check	Observed result
Build / static checking	TypeScript build and `tsc --noEmit` passed.
Unit tests	23 tests passed. 4 Secret Gate socket integration tests were skipped because this sandbox returned `EPERM` for Unix-domain socket creation.
Coverage	12.15% statements, 16.62% branches, 8.73% functions, 13.25% lines. This is low and should be stated plainly.
Packaging	`npm pack --dry-run` showed runtime artifacts and metadata only after a new npm ignore policy; scratch files and logs are excluded.
Dependency audit	The dependency installation reported zero vulnerabilities. A direct advisory query was blocked by the environment DNS/network restriction, so it is not an independent audit result.

> **Important:** automated end-to-end MCP Watch coverage is still missing. The relay was hardened structurally, but a demonstration should be performed against the specific MCP client/server combination you intend to present.

6. Known limitations and next work

Do not overclaim

- Guard does not secure an application already exposed by another interface, and it does not replace application authentication or authorization.
- MCP Watch detects selected suspicious path patterns; it cannot prove a path operation was blocked, nor can it inspect every possible tool argument schema.
- Secret Gate moves secrets out of a regular file only after successful startup and explicit authorization. It does not make secrets inaccessible to the same operating-system user.
- Linux-like Unix socket behavior is the best-tested intent. Windows named-pipe behavior is not equivalently integration-tested.
- Coverage is insufficient for a production security claim, especially proxy lifecycle, child shutdown, CLI failures, MCP stream behavior, and real socket behavior.

Highest-value next steps

1. Add isolated HTTP and WebSocket proxy integration tests.
2. Add real MCP client/server fixtures covering chunks, multiple messages, backpressure, child failure, and warning output.
3. Run Secret Gate tests on a normal machine with `AIRLOCK_TEST_UNIX_SOCKETS=1`.
4. Add CI for build, lint, test, coverage, and npm-package contents.
5. Consider enforcement only after a protocol-level policy and compatibility matrix exist.

7. Demonstration message

Opening: “Airlock is a local security prototype focused on three common developer trust assumptions: localhost is private, AI tool calls stay in scope, and a dotenv file is harmless.”

Guard: show one accepted localhost request, then hostile Host and Origin requests being dropped. State that this is the most demonstrably enforced component.

Secret Gate: use fake credentials only. Show an explicit request for one key and show that deletion requires the separate `--delete-source` flag.

MCP Watch: call it an observability layer, not a blocker. Show its stderr warning/log for a suspicious path only after validating it with the chosen MCP server.

> **Closing:** “The value of Airlock at this stage is explicit boundaries and honest behavior: block where the input is clear, monitor where enforcement would be disruptive, and document what the system cannot promise.”

For exact commands, expected outputs, and recovery actions, use `DEMO_RUNBOOK.md` in the repository.

This report intentionally distinguishes design intent from demonstrated implementation. It is the safer and more credible document to present alongside the 22-page master project document.